

LAPORAN TUGAS BESAR 1
IF2224 Teori Bahasa Formal dan Automata
Semester II 2025/2026

Arion Compiler Milestone 3 - Semantic Analysis



Stevanus Agustaf Wongso	13524020
Renuno Yuqa Frinardi	13524080
Valentino Daniel Kusumo	13524104
Michael James Liman	13524106

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI.....	1
BAB I	
DASAR TEORI.....	3
1. Semantic Analysis.....	3
1.1. Pengertian Semantic Analysis.....	3
1.2. Tujuan Semantic Analysis.....	3
1.3. Pemeriksaan dalam Semantic Analysis.....	3
2. Abstract Syntax Tree (AST).....	4
2.1. Pengertian AST.....	4
2.2. Fungsi AST Pada Compiler.....	4
2.3. Komponen Umum AST.....	4
2.4. Traversal AST.....	5
3. Symbol Table.....	5
3.1. Pengertian Symbol Table.....	5
3.2. Fungsi Symbol Table.....	5
3.3. Operasi dalam Symbol Table.....	5
4. L-Attributed Grammar.....	6
4.1. Pengertian L-Attributed Grammar.....	6
4.2. Peran L-Attributed Grammar Pada Semantic Analysis.....	6
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	7
1. Perancangan.....	7
1.1. Gambaran Umum Sistem.....	7
1.2. Perancangan Semantic Action.....	7
1.3. Perancangan Symbol Table.....	12
1.4. Perancangan Predefined Identifier.....	14
1.5. Perancangan Type Compatibility Rule.....	15
2. Implementasi.....	17
2.1. Arsitektur File.....	17
2.2. Struktur Data.....	19
2.3. Implementasi Abstract.....	23
2.4. Implementasi Symbol Table.....	25
2.5. Implementasi Analyzer.....	26
BAB III	
PENGUJIAN.....	29
1. Test Case 1.....	29
2. Test Case 2.....	30
3. Test Case 3.....	33
4. Test Case 4.....	36
5. Test Case 5.....	38
6. Test Case 6.....	41
BAB IV	
KESIMPULAN DAN SARAN.....	45
1. Kesimpulan.....	45

2. Saran.....	45
LAMPIRAN.....	46
REFERENSI.....	47

BAB I

DASAR TEORI

1. Semantic Analysis

1.1. Pengertian Semantic Analysis

Semantic Analysis adalah tahap pada proses kompilasi yang memeriksa arti program setelah program melewati analisis leksikal dan analisis sintaksis. Pada tahap ini, *compiler* memastikan bahwa konstruksi program digunakan secara konsisten dengan definisi bahasa pemrograman, misalnya deklarasi variabel, tipe data, pemanggilan fungsi, dan ruang lingkup identifier. Dengan demikian, semantic analysis menangani kesalahan yang tidak dapat ditangkap hanya oleh lexical analyzer dan parser berbasis *context-free grammar*.

Program dapat benar secara sintaksis, tetapi tetap salah secara semantik. Misalnya, statement assignment dapat memiliki bentuk yang benar, tetapi nilai yang diberikan tidak sesuai dengan tipe variabel tujuan, seperti contoh di bawah.

```
int x;  
x = "hello";
```

Contoh tersebut mengikuti bentuk assignment yang sah, tetapi tidak valid secara semantik karena variabel *x* bertipe *int*, sedangkan nilai yang diberikan berupa *string*.

1.2. Tujuan Semantic Analysis

- Memastikan setiap identifier seperti variabel, fungsi, parameter, atau konstanta sudah dideklarasikan sebelum digunakan.
- Memeriksa kesesuaian tipe data dalam assignment, operasi aritmetika, operasi logika, pemanggilan fungsi, dan return statement.
- Memastikan aturan scope dipatuhi, sehingga identifier hanya digunakan pada ruang lingkup yang valid.
- Memeriksa deklarasi ulang dalam scope yang sama agar tidak terjadi konflik nama.
- Mengumpulkan informasi semantik yang diperlukan oleh tahap berikutnya, seperti intermediate code generation, optimasi, dan code generation.

1.3. Pemeriksaan dalam Semantic Analysis

Beberapa aktivitas utama dalam semantic analysis meliputi:

1. Name Checking

Pemeriksaan apakah nama variabel, fungsi, konstanta, atau identifier lainnya sudah dideklarasikan dan digunakan sesuai aturan.

2. Type Checking

Pemeriksaan kesesuaian tipe data dalam ekspresi, assignment, pemanggilan

fungsi, return statement, dan operasi lainnya.

3. Scope Checking

Pemeriksaan ruang lingkup identifikasi untuk memastikan bahwa suatu variabel hanya digunakan di area yang sesuai.

4. Function Checking

Pemeriksaan deklarasi fungsi, tipe return, jumlah parameter, tipe parameter, dan pemanggilan fungsi.

5. Control Flow Checking

Pemeriksaan alur kontrol program, misalnya memastikan bahwa fungsi non-void memiliki return statement.

6. Attribute Evaluation

Evaluasi atribut pada node-node dalam AST untuk memperoleh informasi tambahan, seperti tipe data, nilai konstanta, atau lokasi memori.

2. Abstract Syntax Tree (AST)

2.1. Pengertian AST

Abstract Syntax Tree (AST) adalah representasi pohon dari struktur sintaksis abstrak suatu program. Setiap node pada AST mewakili konstruksi penting dalam *source code*, seperti deklarasi, statement, ekspresi, operator, literal, atau pemanggilan fungsi. AST disebut abstrak karena tidak harus menyimpan seluruh detail sintaksis, seperti tanda titik koma, tanda kurung yang hanya berfungsi sebagai pengelompokan, atau detail turunan grammar yang tidak diperlukan untuk analisis lanjutan.

Sebagai contoh, ekspresi $a + b * c$ direpresentasikan dengan operator $+$ sebagai akar ekspresi, sementara operasi $b * c$ menjadi subtree karena perkalian memiliki prioritas lebih tinggi daripada penjumlahan.

2.2. Fungsi AST Pada Compiler

- Menjadi representasi internal program setelah *parsing*.
- Memudahkan traversal program untuk *semantic analysis*.
- Menyimpan struktur operator, prioritas operasi, statement, deklarasi, dan ekspresi secara eksplisit.
- Menjadi dasar untuk menghasilkan *intermediate representation* atau *intermediate code*.
- Mendukung optimasi dan transformasi kode, misalnya *constant folding* dan *refactoring*.

2.3. Komponen Umum AST

- **Program Node:** Node akar yang merepresentasikan keseluruhan program atau *compilation unit*.
- **Declaration Node:** Node yang merepresentasikan deklarasi variabel, fungsi, konstanta, class, atau tipe data.
- **Statement Node:** Node yang merepresentasikan instruksi program, seperti if, while, for, return, atau assignment.

- **Expression Node:** Node yang merepresentasikan ekspresi aritmatika, logika, relasional, atau pemanggilan fungsi.
- **Identifier Node:** Node yang merepresentasikan nama variabel, fungsi, parameter, atau entitas bernama lainnya.
- **Literal Node:** Node yang merepresentasikan nilai literal seperti angka, karakter, boolean, atau string.

2.4. Traversal AST

Traversal adalah proses menelusuri node-node pada AST. Traversal dipakai untuk melakukan pengecekan semantik, pembentukan symbol table, evaluasi ekspresi, optimasi, dan pembentukan *intermediate code*. Beberapa cara traversal node-node AST adalah sebagai berikut:

- **Preorder Traversal:** Node saat ini diproses terlebih dahulu, kemudian anak-anaknya. Berguna ketika informasi dari parent perlu diketahui lebih dahulu.
- **Postorder Traversal:** Anak-anak node diproses terlebih dahulu, kemudian parent. Berguna untuk evaluasi ekspresi karena nilai operand harus diketahui sebelum operator dievaluasi.
- **Depth-first Traversal:** Traversal menyusuri cabang pohon sedalam mungkin sebelum berpindah ke cabang lain. Pola ini banyak digunakan dalam evaluasi atribut dan pemeriksaan struktur program.

3. Symbol Table

3.1. Pengertian Symbol Table

Symbol table adalah struktur data yang digunakan compiler untuk menyimpan informasi tentang nama atau simbol dalam program. Informasi tersebut dapat mencakup nama identifier, kelas symbol, tipe data, scope, ukuran, lokasi deklarasi, parameter fungsi, serta informasi penyimpanan. Symbol table mencatat informasi tentang setiap symbol name dalam program dan biasanya dibentuk atau dilengkapi pada tahap semantic analysis, karena pada tahap tersebut compiler telah memiliki cukup informasi untuk mendeskripsikan suatu nama.

3.2. Fungsi Symbol Table

- Mencatat deklarasi identifier beserta atributnya.
- Mendukung lookup ketika identifier digunakan dalam ekspresi, assignment, atau pemanggilan fungsi.
- Mendeteksi error seperti identifier belum dideklarasikan atau deklarasi ulang dalam scope yang sama.
- Membantu type checking dengan menyediakan informasi tipe identifier.
- Membantu code generation dengan menyimpan informasi ukuran data, offset, atau lokasi memori.

3.3. Operasi dalam Symbol Table

- **Insert:** Menambahkan identifier baru ke symbol table saat deklarasi ditemukan. Jika nama yang sama sudah ada pada scope yang sama, compiler dapat menghasilkan error *redeclaration*.

- **Lookup:** Mencari identifier saat identifier digunakan. Jika tidak ditemukan pada scope aktif maupun scope luar yang valid, compiler menghasilkan error undeclared identifier.
- **Update:** Memperbarui atribut symbol, misalnya status definisi fungsi atau informasi lokasi memori.
- **Delete/Pop Scope:** Menghapus atau menonaktifkan symbol ketika compiler keluar dari scope tertentu, khususnya pada implementasi stack of symbol tables.

4. L-Attributed Grammar

4.1. Pengertian L-Attributed Grammar

L-Attributed Grammar adalah jenis attribute grammar yang membatasi ketergantungan inherited attribute agar dapat dievaluasi dari kiri ke kanan. Untuk produksi $A \rightarrow X_1 X_2 \dots X_n$, inherited attribute dari X_i hanya boleh bergantung pada inherited attribute milik A dan atribut dari simbol-simbol X_1 sampai X_{i-1} yang berada di sebelah kiri X_i . Definisi ini mencegah ketergantungan dari kanan ke kiri sehingga evaluasi atribut dapat dilakukan dalam urutan *depth-first* atau *one-pass left-to-right*.

4.2. Peran L-Attributed Grammar Pada Semantic Analysis

L-Attributed Grammar membantu compiler mendefinisikan proses pemeriksaan semantik secara formal. Informasi seperti tipe data, scope aktif, dan entry symbol table dapat dialirkan melalui atribut. Karena ketergantungannya dibatasi dari kiri ke kanan, semantic action dapat ditempatkan secara sistematis dalam proses parsing atau traversal. Konsep ini penting dalam syntax-directed translation, terutama ketika compiler perlu membangun symbol table dan melakukan type checking secara bertahap.

BAB II

PERANCANGAN DAN IMPLEMENTASI

1. Perancangan

1.1 Gambaran Umum Sistem

Pada *milestone* 3 ini, program menambahkan tahap *Semantic Analysis* setelah tahapan sebelumnya, yaitu *Lexical Analysis*. Secara umum, alur kerja program dapat digambarkan sebagai berikut:

Source code (.txt) → Tokens → Parse Tree → Decorated AST

Tahap Semantic Analysis terdiri atas 2 proses utama, yaitu:

1. Konversi Parse Tree Menjadi AST

Proses ini menggunakan *syntax-directed transition* (SDT) dengan L-Attributed Grammar untuk membuang *node* sintaksis yang tidak diperlukan lagi dan membangun struktur AST.

2. Dekorasi AST

Pada tahap ini, AST ditelusuri menggunakan fungsi visit dengan metode Depth-First Search (DFS). Proses tersebut mencakup registrasi dan pencarian identifier pada Symbol Table, serta validasi tipe data pada setiap *expression* dan *statement*.

Spesifikasi proses:

- **Input:** Parse Tree
- **Output:** Decorated AST beserta Symbol Table
- **Algoritma:** L-Attributed Grammar

1.2 Perancangan Semantic Action

Proses konversi Parse Tree menjadi Abstract Syntax Tree (AST) menggunakan Syntax Directed Translation (SDT) dengan L-Attributed Grammar. Setiap node pada Parse Tree diproses secara bottom-up, yaitu atribut synthesized dari child dinaikkan ke parent, lalu parent membangun node AST baru dari atribut tersebut. Node sintaksis yang tidak membawa makna semantik (misal: BEGIN, END, semicolon, tanda kurung, wrapper non-terminal seperti <simple-expression> yang hanya punya 1 child) dibuang.

1. Program & Deklarasi

Nama Non-Terminal	Tipe Node AST	Keterangan Semantic Action
<program>	Program	Node root. value = nama program dari ident di <program-header> Children: [Declarations, Compound]

<program-header>	(dibuang)	Tidak menghasilkan node AST sendiri. ident disintesis ke parent sebagai value dari Program
<declaration-part>	Declarations	Container seluruh deklarasi. Setiap child <var-declaration>, <const-declaration>, <type-declaration>, <subprogram-declaration> diproses dan hasilnya di-flatten ke dalam Declarations
<var-declaration>	VarDecl (<i>jamak</i>)	Setiap ident dalam <identifier-list> di tiap <var-item> diperluas menjadi node VarDecl tersendiri. value = nama variabel, child = node tipe dari <type>
<const-declaration>	ConstDecl	Tiap <const-item> menghasilkan satu ConstDecl. value = nama konstanta, child = node ekspresi nilai konstanta
<type-declaration>	TypeDecl	Tiap <type-item> menghasilkan satu TypeDecl. value = nama tipe, child = node tipe
<subprogram-declaration>	SubprogramDecl	value = nama subprogram (ident pertama setelah keyword), extra = "procedure" atau "function", child = Compound body
<type>	(delegasi)	Wrapper; mendelegasi ke child. Jika child ident → TypeRef . Jika <array-type> / <record-type> / <enumerated> / <range> → builder masing-masing
<array-type>	ArrayType	Children: [index-type (dari <range> atau ident), element-type (dari <type>)]
<record-type>	RecordType	Children: VarDecl untuk tiap field. Setiap ident dalam <identifier-list> per <field-part> diperluas jadi VarDecl tersendiri
<enumerated>	EnumType	Children: node Var untuk tiap ident enumerator
<range>	RangeType	Children: [ekspresi batas bawah, ekspresi batas atas], keduanya dibangun dari <constant>

ident(name) dalam konteks tipe	TypeRef	Node daun. value = nama tipe yang dirujuk
<identifier-list>	(dibuang)	Tidak menghasilkan node AST sendiri. Setiap ident di dalamnya diperluas ke parent masing-masing
<var-item>, <const-item>, <type-item>	(dibuang)	Wrapper sintaksis. Token seperti colon, semicolon, eql dibuang, isi semantiknya diproses oleh parent

2. Statement

Nama Non-Terminal	Tipe Node AST	Keterangan Semantic Action
<compound-statement>	Compound	Children: semua statement dari <statement-list>. Token semicolon dan beginsy/endsy diabaikan
<statement-list>	(dibuang)	Wrapper; setiap child diproses oleh buildStatement dan hasilnya diangkat ke Compound
<statement>	(delegasi)	Wrapper; didispatch berdasarkan label child ke builder statement spesifik
<assignment-statement>	Assign	Children: [target, value]. Target diambil dari ident atau <variable>. Value dari <expression>. Token becomes diabaikan
<if-statement>	If	Children: [cond, then-stmt, (else-stmt)?]. Cond dari <expression>, then/else dari <statement>. Token ifsy, then, else, endsy diabaikan
<while-statement>	While	Children: [cond, body]. Cond dari <expression>, body dari <compound-statement>. Token whilesy, dosy diabaikan
<for-statement>	For	value = nama variabel kontrol (ident), extra = "to" atau "downto". Children: [start-expr, end-expr, body]. Token forsy, becomes, tosy/downtosy, dosy diabaikan

3. Expression

Nama Non-Terminal	Tipe Node AST	Keterangan Semantic Action
<expression>	BinOp / (<i>pass-through</i>)	Jika ada operator relasional (=, <>, <, <=, >, >=) di antara dua <simple-expression>, menghasilkan BinOp dengan value = simbol operator dan children [left, right]. Jika hanya satu <simple-expression>, node-nya diteruskan langsung (tidak dibungkus)
<simple-expression>	BinOp / UnaryOp / (<i>pass-through</i>)	Jika ada operator (+, -, or), membangun rantai BinOp kiri-asosiatif dari <term>-<term>. Jika ada tanda unary di awal, membungkus dengan UnaryOp (value="+" atau "-"). Jika

		hanya satu term, node-nya diteruskan langsung
<term>	BinOp / (<i>pass-through</i>)	Jika ada operator (*, /, div, mod, and), membangun rantai BinOp kiri-asosiatif dari <factor>-<factor>. Jika hanya satu faktor, node-nya diteruskan langsung
<factor>	(<i>delegasi</i>)	Mendispach berdasarkan child: intcon → Number ; realcon → RealNumber ; charcon → CharLit ; string → StringLit ; ident("true"/"false") → BoolLit ; ident(name) → Var ; notsy + <factor> → UnaryOp ("not"); lparent <expression> rparent → rekursi ke buildExpression; <variable> → buildVariable; <procedure/function-call> → buildProcFuncCall
<constant>	(<i>sama dengan factor</i>)	Diperlakukan seperti <factor>. Jika diawali minus, hasilnya dibungkus dalam UnaryOp (""). Token plus diabaikan
<parameter-list>	(<i>dibuang</i>)	Wrapper; tiap child <expression> diproses dan hasilnya diangkat sebagai argumen ke ProcCall

4. Literal & Identifiers

Token	Tipe Node AST	Keterangan
intcon(n)	Number	Node daun. value = representasi string dari integer
realcon(n)	RealNumber	Node daun. value = representasi string dari real
charcon(c)	CharLit	Node daun. value = karakter tunggal (tanpa tanda kutip)
string(s)	StringLit	Node daun. value = isi string (tanpa tanda kutip)

ident("true"/"false")	BoolLit	Node daun. value = "true" atau "false"
ident(name)	Var	Node daun. value = nama identifier

1.3 Perancangan Symbol Table

Symbol Table merupakan struktur data yang digunakan untuk menyimpan dan menelusuri informasi setiap *identifier* dalam program setelah parse tree dikonversi menjadi AST yang lebih sederhana dan hanya memuat node bermakna. Pada Arion Compiler, Symbol Table juga menjadi dasar dekorasi AST, di mana semantic analysis *visitor* menganotasi node *identifier* dengan informasi seperti indeks entry tabel simbol, tipe data, level leksikal, dan indeks blok, sehingga menghasilkan decorated AST yang mendukung pengecekan semantik dan proses compiler selanjutnya.

Komponen	Nama Variabel	Fungsi
Identifier Table	tab	Menyimpan entry setiap identifier, termasuk reserved word, predefined identifier, variabel, konstanta, tipe, prosedur, fungsi, dan program.
Block Table	btab	Menyimpan informasi setiap blok atau scope. Field last menunjuk identifier terakhir dalam linked list identifier pada blok tersebut.
Array Table	atab	Disiapkan untuk menyimpan informasi tipe array, seperti tipe indeks, tipe elemen, batas bawah, batas atas, ukuran elemen, dan ukuran total.
Display Stack	display	Menyimpan indeks btab yang aktif pada setiap lexical level, sehingga lookup dapat dilakukan dari scope terdalam ke scope terluar.
Lexical Level	level	Menandai kedalaman scope aktif. Level 0 digunakan untuk scope global, sedangkan level lebih tinggi digunakan untuk scope lokal atau subprogram.

Tabel simbol dirancang menggunakan tiga tabel utama, yaitu **tab**, **btab**, dan **atab**, serta satu struktur pendukung bernama display. Struktur ini memiliki deklarasi, blok program, subprogram, tipe dasar, tipe array, dan scope

bertingkat. Setiap *identifier* disimpan sebagai TabEntry yang memuat informasi seperti nama, jenis objek, tipe data, level leksikal, alamat atau nilai, serta hubungan ke identifier sebelumnya dalam scope yang sama. Dengan cara ini, *identifier* dalam satu blok disimpan sebagai linked list melalui field link, sementara entry terakhir pada blok disimpan di btab[blockIdx].last agar pencarian dapat dimulai dari deklarasi terbaru.

Atribut	Tipe	Deskripsi
id	string	Nama identifier yang terdaftar di tabel simbol.
link	int	Indeks entry sebelumnya dalam scope yang sama; membentuk linked list antar entry pada satu blok.
obj	int	Kode kelas objek berdasarkan ObjClass, seperti CONSTANT, VARIABLE, TYPE_DECL, PROCEDURE, FUNCTION, atau PROGRAM.
type	int	Kode tipe berdasarkan TypeCode, seperti TC_INTEGER, TC_REAL, TC_BOOLEAN, TC_CHAR, TC_STRING, TC_ARRAY, atau TC_RECORD.
ref	int	Referensi tambahan ke tabel lain. Field ini direncanakan untuk menghubungkan entry array ke atab atau entry subprogram ke btab.
nrm	bool	Penanda mode parameter. Nilai true menyatakan parameter normal/by-value, sedangkan false dapat dipakai untuk by-reference.
lev	int	Level leksikal tempat identifier dideklarasikan.
adr	int	Alamat, offset, atau nilai konstanta sederhana yang dapat digunakan pada tahap lanjutan.

Pengelolaan scope dilakukan menggunakan btab, display, dan level. Scope global berada pada level 0. Saat compiler masuk ke *procedure* atau *function*, level dinaikkan dan blok baru dibuat. display[level] akan menunjuk ke blok yang sedang aktif. Ketika keluar dari blok, level diturunkan sehingga scope kembali ke blok luar.

Proses pencarian identifier menggunakan strategi deepest-scope-first. Compiler mulai mencari dari scope yang paling dalam, lalu bergerak ke scope yang lebih luar hingga level 0. Pada setiap level, compiler menelusuri linked list identifier melalui btab[blockIdx].last dan field link. Dengan cara ini, deklarasi lokal dapat menutupi deklarasi global yang memiliki nama sama, sedangkan identifier yang tidak ditemukan dianggap belum dideklarasikan.

Rancangan ini juga mendukung deteksi redeclaration. Sebelum identifier baru dimasukkan, compiler memeriksa apakah nama yang sama sudah ada pada scope aktif. Jika ditemukan pada scope yang sama, compiler melaporkan error redeclaration. Namun, jika nama tersebut hanya ada pada scope luar, identifier masih dianggap valid karena *lexical scoping* memperbolehkan *shadowing*.

Reserved word dan predefined identifier dimasukkan saat inisialisasi Symbol Table. Kata kunci seperti program, begin, end, var, function, dan procedure disimpan sebagai simbol bawaan compiler. Tipe bawaan seperti **integer**, **real**, **boolean**, **char**, dan **string** juga dimasukkan sebagai deklarasi tipe. Selain itu, *identifier* bawaan seperti **True**, **False**, **Readln**, dan **Writeln** ditambahkan agar dapat dikenali selama semantic analysis.

1.4 Perancangan Predefined Identifier

- **Daftar Wajib**

Nama Tipe	Keterangan
Real	Memiliki type Real
Integer	Memiliki type Integer
Char	Memiliki type Char
Boolean	Memiliki type Boolean
String	Memiliki type String
True	Memiliki type Boolean dengan nilai <i>true</i>
False	Memiliki type Boolean dengan nilai <i>false</i>

- **Tambahan**

Menambahkan I/O routines predefined, yaitu **Readln** dan **Writeln** (diimplementasikan sebagai PROCEDURE dengan predefined flag)

- **Representasi Internal**

Setiap predefined dimasukkan ke tab melalui enter() dengan field:

- obj = ObjClass::TYPE_DECL untuk tipe dasar type = TC_*
- obj = ObjClass::CONSTANT untuk True/False, type = TC_BOOLEAN, adr/nilai ordinal bila perlu.
- obj = ObjClass::PROCEDURE/FUNCTION untuk rutin I/O, ref bisa menunjuk ke block parameter.
- Batas pemisah predefined disimpan di predefinedCutoff() sehingga pemanggilan dapat mendeteksi entry predefined.

- **Lokasi inisialisasi**

inisialisasi dilakukan saat *SymbolTable* dibuat

1.5 Perancangan Type Compatibility Rule

Type Compatibility Rule menentukan kapan 2 tipe dianggap *compatible* sehingga dapat digunakan bersama dalam *expression* atau proses *assignment*. Aturan ini dibagi menjadi beberapa kategori sesuai dengan konteks penggunaannya.

1. Tipe Dasar

Nama Tipe	Keterangan
Integer	intcon memiliki type Integer
Real	realcon memiliki type Real
Char	charcon memiliki type Char
Boolean	term dan expression yang diproduksi dari notsy , andsy , dan orsy memiliki type Boolean expression yang diproduksi dari relational-operator memiliki type Boolean
String	string memiliki type String
Subrange	Type dari Node Range ditentukan oleh type dari Constant di dalamnya. lower bound tidak boleh $\leq$ upper bound dan tidak boleh bertipe real .
Enumerated	Type dari Node Enumerated ditentukan oleh type dari Ident di dalamnya.

2. Tipe Terstruktur

Nama Tipe	Aturan
Array	Memiliki bentuk: array [T1] of T2. T1 (index type) harus berupa <i>simple type</i> dan tidak boleh Real. T2 (element type) dapat berupa tipe apapun termasuk array atau record
Record	Field di dalam record dapat memiliki tipe apapun. Named record (didefinisikan dalam type-declaration) kompatibel berdasarkan nama tipe. Anonymous record (didefinisikan langsung di var-declaration) bersifat <i>incompatible</i> meskipun strukturnya identik

3. Aturan *compatible*

2 tipe T1 dan T2 dikatakan *compatible* jika memenuhi salah satu dari kondisi berikut:

- 1) Kedua tipe adalah tipe yang sama
- 2) 1 tipe adalah *subrange* dari yang lain atau kedua tipe adalah *subrange* dari tipe yang sama
- 3) Kedua tipe memiliki String dan panjang yang sama

4. Aturan *assignment-compatible*

Sebuah tipe T2 adalah *assignment-compatible* dengan tipe T1 jika memenuhi salah satu dari kondisi berikut:

- 1) T1 adalah Real dan T2 adalah Integer
- 2) T1 dan T2 adalah *compatible*, bertipe Integer/Boolean/Char dan nilai T2 merupakan subset dari nilai T1
- 3) T1 dan T2 adalah *compatible* dan bertipe String

5. Aturan Tipe untuk Kontrol Program

Operasi Kontrol	Aturan
if-statement	Ekspresi kondisi harus bertipe Boolean
while-statement	Ekspresi kondisi harus bertipe Boolean
for-statement	Variabel kontrol harus bertipe ordinal (Integer, Char, atau subrange). Ekspresi batas awal dan akhir harus <i>assignment-compatible</i> dengan variabel kontrol
Operator not	Operand harus bertipe Boolean, hasil bertipe Boolean
Operator and / or	Kedua operand harus bertipe Boolean, hasil bertipe Boolean
Operator Relasional (=, <>, <, >, <=, >=)	Kedua operand harus <i>compatible</i> , hasil bertipe Boolean
Operator aritmatika (+, -, *, /)	Operand harus bertipe numerik (Integer / Real). Jika salah satu Real, hasil bertipe Real. Jika keduanya Integer, hasil bertipe Integer
Operator div / mod	Kedua operand harus bertipe Integer, hasil bertipe Integer
Operator + pada String (<i>concatenate</i>)	Kedua operand harus bertipe String, hasil bertipe String

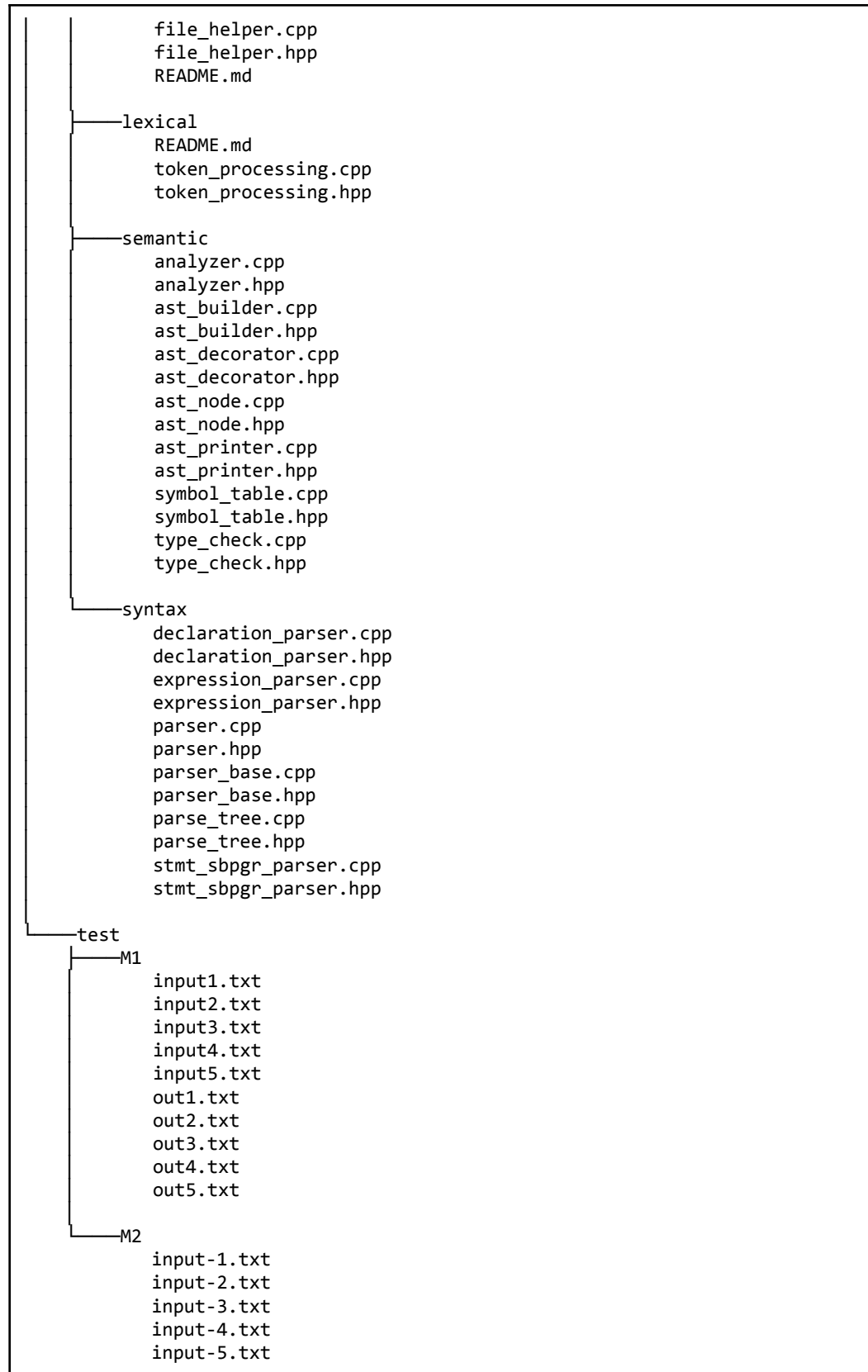
2. Implementasi

2.1 Arsitektur File

Program dibagi ke dalam beberapa modul dengan tanggung jawab yang terpisah.

File	Peran
src/semantic/analyzer	Implementasi utama semantic analyzer: memproses deklarasi, statement, ekspresi, pemanggilan prosedur/fungsi, resolusi identifier, serta inference dan validasi tipe.
src/semantic/ast_builder	Implementasi transformasi parse tree ke AST: membangun node Program, Declarations, Compound, Assign, BinOp, UnaryOp, Var, ProcCall/FuncCall, tipe, dan subprogram.
src/semantic/ast_decorator	Implementasi dekorasi AST: lookup simbol, isi typeName/tabIndex/lev/blockIndex, membedakan procedure dan function, serta anotasi node literal dan operator.
src/semantic/ast_node	Implementasi helper dasar AST: membuat node, menambahkan child, dan mengubah AstKind menjadi nama string.
src/semantic/ast_printer	Implementasi printer AST berbentuk tree ASCII, termasuk format node, label child, dan tampilan anotasi semantic.
src/semantic/symbol_table	Implementasi symbol table: inisialisasi reserved word dan predefined identifier, pengelolaan block/scope, lookup case-insensitive, enter/setRef, serta dump tab/btab/atab.
src/semantic/type_check	Implementasi aturan tipe yang stateless: klasifikasi tipe, kompatibilitas assignment, hasil operator biner dan relasional, validasi range/index array, dan pembentukan tipe dasar.





2.2 Struktur Data

1. Token

Token dihasilkan oleh lexer dan menjadi input parser dengan struktur:

```
struct Token {  
    string type;  
    string value;  
};
```

- type menyimpan jenis token (mis. "ident", "intcon", "plus")
- value menyimpan nilai literalnya (kosong untuk operator dan keyword)

2. ParseNode

Node parse tree direpresentasikan sebagai:

```
struct ParseNode {  
    string label;  
    vector<shared_ptr<ParseNode>> children;  
  
    explicit ParseNode(const string& lbl) : label(lbl) {}  
};  
  
using NodePtr = shared_ptr<ParseNode>;
```

- Setiap node dapat memiliki anak sejumlah apapun sesuai produksi grammar-nya. Node terminal (token) tidak memiliki anak, sedangkan node non-terminal seperti <program> dan <declaration-part> memiliki anak yang merepresentasikan komponen-komponennya.

3. ParseError

Informasi error saat parsing disimpan sebagai:

```
struct ParseError {  
    string message;  
    int tokenIndex;  
    string tokenType;  
    string tokenValue;  
};
```

- message berisi deskripsi error
- tokenIndex menunjuk posisi token penyebab error
- tokenType/tokenValue menyimpan detail token tersebut untuk keperluan pelaporan

4. AstKind

Tag enum untuk menentukan jenis setiap node AST, digunakan untuk dispatch visitor tanpa dynamic_cast:

```
enum class AstKind {
    Program, Declarations,
    VarDecl, ConstDecl, TypeDecl,
    SubprogramDecl, SubprogramHeader,
    Block, Compound, Empty,
    Assign, If, While, For, Repeat, Case,
    ProcCall, FuncCall,
    BinOp, UnaryOp,
    Var, Number, RealNumber, CharLit, StringLit, BoolLit,
    IndexAccess, FieldAccess,
    TypeRef, ArrayType, RecordType, EnumType, RangeType
};
```

5. AstNode

Node AST hasil konversi dari Parse Tree direpresentasikan sebagai:

```
struct AstNode {
    AstKind kind;
    string value;
    string extra;
    vector<shared_ptr<AstNode>> children;

    string typeName;
    int tabIndex = -1;
    int lev = -1;
    int blockIndex = -1;
    bool predefined = false;
};

using AstNodePtr = shared_ptr<AstNode>;
```

- value menyimpan nama identifier, nilai literal, atau simbol operator
- extra menyimpan informasi tambahan seperti arah for-loop ("to"/"downto") atau jenis subprogram ("procedure"/"function")
- Field typeName, tabIndex, lev, blockIndex, dan predefined diisi pada tahap dekorasi AST.

6. ObjClass

Enum yang menandai kelas objek setiap identifier dalam Symbol Table:

```
enum class ObjClass {
    UNDEFINED = 0,
    CONSTANT = 1,
    VARIABLE = 2,
    TYPE_DECL = 3,
    PROCEDURE = 4,
    FUNCTION = 5,
    PROGRAM = 6
};
```

7. TypeCode

Enum yang mengkodekan tipe dasar untuk disimpan di field type pada TabEntry:

```
enum TypeCode {
    TC_NOTYPE = 0,
    TC_INTEGER = 1,
    TC_REAL = 2,
    TC_BOOLEAN = 3,
    TC_CHAR = 4,
    TC_STRING = 5,
    TC_ARRAY = 6,
    TC_RECORD = 7
};
```

8. TabEntry

Satu entry dalam identifier table. Setiap identifier yang terdaftar memiliki satu TabEntry:

```
struct TabEntry {
    string id;
    int link;
    int obj;
    int type;
    int ref;
    bool nrm;
    int lev;
    int adr;
};
```

- link membentuk linked list antar identifier dalam satu scope
- ref menunjuk ke btab untuk prosedur/fungsi atau ke atab untuk array
- nrm membedakan parameter by-value (*true*) dan by-reference (*false*).

9. BtabEntry

Satu entry dalam block table. Setiap scope (program utama, prosedur, fungsi) memiliki satu BtabEntry:

```
struct BtabEntry {
    int last;
    int lpar;
    int psze;
    int vsze;
};
```

- last adalah indeks tab dari identifier terakhir yang terdaftar di block ini
- lpar menandai batas antara parameter dan variabel lokal.

10. AtabEntry

Satu entry dalam array table. Setiap tipe array yang dideklarasikan memiliki satu AtabEntry:

```
struct AtabEntry {
    int xtyp;
    int etyp;
    int eref;
    int low;
    int high;
    int elsz;
    int size;
};
```

- xtyp adalah tipe index
- etyp adalah tipe elemen
- eref menunjuk ke atab/btab jika elemen bertipe structured
- low dan high adalah batas index
- elsz dan size menyimpan ukuran elemen dan total array.

11. SymbolTable

Kelas utama yang mengelola seluruh tabel simbol dengan model linked-list per scope dan display array untuk resolusi lexical scoping:

```
class SymbolTable {
    vector<TabEntry> tab;
    vector<BtabEntry> btab;
    vector<AtabEntry> atab;
    vector<int> display;
    int level;
    int predefinedEnd;
};
```

- display[lev] menyimpan indeks btab yang aktif di lexical level lev, memungkinkan pencarian identifier dimulai dari scope terdalam ke terluar
- predefinedEnd adalah *cutoff* index, semua entry di bawah nilai ini adalah reserved word atau identifier predefined.

12. SemanticType

Struct yang menampung informasi metadata dari type pada semantic analyzer. Attribut SemanticType digunakan untuk type-checking logic.

```
struct SemanticType {
    int type = TC_NOTYPE;
    string name;
    bool anonymous = false;
    int ref = 0;
```

```

int low = 0;
int high = 0;
int size = 1;
bool isSubrange = false;
bool hasRange = false;
int len = 0;
bool hasLen = false;
};

```

- type adalah enumerate TC_INTEGER, TC_REAL, TC_RECORD, dll.
- name merupakan string sebagai nama tipenya seperti integer, real, dll.
- ref adalah index pada symbol table entry.
- isSubrange menandai tipe sebagai subrange.
- low dan high merupakan batas numeric untuk subrange.
- len merupakan metadata yang mendeklarasikan panjang string.
- hasLen mendeklarasikan apakah variabel len diketahui.

2.3 Implementasi Abstract

Konversi Parse Tree ke AST dilakukan oleh kelas AstBuilder melalui metode build() sebagai entry point:

```

AstNodePtr AstBuilder::build(const NodePtr& parseRoot){
    errors.clear();
    if (!parseRoot){
        reportError("Parse tree is null");
        return nullptr;
    }
    return buildProgram(parseRoot);
}

```

Proses konversi berjalan secara rekursif bottom-up: setiap metode build* memproses child-nya terlebih dahulu, lalu membangun node AST baru dari atribut yang disintesis ke atas. Node-node sintaksis yang tidak membawa makna semantik, seperti `beginsy`, `endsy`, `semicolon`, `colon`, `lparent`, `rparent`, serta non-terminal wrapper yang hanya punya 1 child semantik tidak diikutsertakan dalam AST.

- **Flattening identifier list**

Satu `<var-item>` yang memiliki beberapa identifier diperluas menjadi beberapa node `VarDecl` terpisah, masing-masing berbagi pointer ke node tipe yang sama :

```

for (auto& id : identList->children) {
    if (!startsWith(id->label, "ident(")) continue;
    auto vd = makeAst(AstKind::VarDecl,
        extractInside(id->label));
    if (typeAst) addAstChild(vd, typeAst);
    addAstChild(group, vd);
}

```

```
}
```

- **Pass-through pada expression**

Non-terminal `<expression>`, `<simple-expression>`, dan `<term>` hanya menghasilkan node `BinOp` jika memiliki lebih dari satu operand. Jika hanya memiliki satu child semantik, node child tersebut langsung diteruskan ke parent tanpa pembungkus:

```
AstNodePtr AstBuilder::buildExpression(const NodePtr& node) {
    AstNodePtr left = buildSimpleExpression(node->children[0]);
    for (size_t i = 1; i + 1 < node->children.size(); i++) {
        auto& opNode = node->children[i];
        if (!isOpToken(opNode->label)) continue;
        auto right = buildSimpleExpression(node->children[i +
            1]);
        auto bin = makeAst(AstKind::BinOp,
            opSymbol(opNode->label));
        addAstChild(bin, left);
        addAstChild(bin, right);
        left = bin;
        i++;
    }
    return left; // pass-through jika tidak ada operator
}
```

- Node `BinOp` dibangun dengan asosiativitas kiri, setiap operator baru membungkus akumulasi kiri sebagai child pertamanya.

- **Dekorasi AST**

Setelah AST terbentuk, `AstDecorator` menelusuri seluruh node menggunakan Depth-First Search untuk mengisi atribut semantik. Untuk setiap node `Var`, `VarDecl`, `ConstDecl`, `ProcCall`, dan sejenisnya, decorator melakukan lookup ke Symbol Table dan menyalin informasi tipe serta lexical level ke node AST:

```
void AstDecorator::annotateFromTab(const AstNodePtr& node,
const string& id) {
    int idx = symbols.lookup(id);
    if (idx >= 0) {
        const auto& e = symbols.getTab()[idx];
        node->tabIndex = idx;
        node->typeName = typeNameFromCode(e.type);
        node->lev = e.lev;
        if (idx < symbols.predefinedCutoff()){
            node->predefined = true;
        }
    }
}
```

- Untuk node *BinOp* dan *UnaryOp*, *typeName* tidak diambil dari Symbol Table melainkan diinferensikan dari tipe operand-nya.
 - Operator relasional dan logika selalu menghasilkan "boolean"
 - Operator div/mod menghasilkan "integer"
 - Operator / menghasilkan "real"
 - Operator +/−/* menghasilkan "real" jika salah satu operand bertipe real dan "integer" jika keduanya integer.

2.4 Implementasi Symbol Table

Implementasi Symbol Table ditempatkan pada modul `src/semantic/symbol_table.hpp` dan `src/semantic/symbol_table.cpp`. File header mendefinisikan enum, struktur data, dan antarmuka kelas `SymbolTable`, sedangkan file implementasi memuat konstruktor, inisialisasi reserved word dan predefined identifier, operasi enter, lookup, pushBlock, popBlock, serta fungsi dump untuk menampilkan isi tabel.

Kelas `SymbolTable` menggunakan `std::vector<TabEntry>` untuk identifier table, `std::vector<BtabEntry>` untuk block table, `std::vector<AtabEntry>` untuk array table, dan `std::vector<int>` untuk display. Pemakaian vector memudahkan akses berbasis indeks, sehingga setiap entry dapat direferensikan menggunakan bilangan bulat. Hal ini sesuai dengan rancangan `tab/btab/atab` yang umum dipakai pada compiler sederhana.

Struktur	Implementasi C++	Peran
Identifier Table	<code>std::vector<TabEntry> tab</code>	Menyimpan seluruh entry identifier dan reserved/predefined symbol.
Block Table	<code>std::vector<BtabEntry> btab</code>	Menyimpan informasi scope atau blok aktif.
Array Table	<code>std::vector<AtabEntry> atab</code>	Menyimpan metadata tipe array jika tipe array digunakan.
Display	<code>std::vector<int> display</code>	Memetakan lexical level ke indeks block table yang sedang aktif.
Level	<code>int level</code>	Mencatat level scope aktif.

Konstruktor `SymbolTable` menginisialisasi tabel dengan membuat sentinel pada `tab[0]`, lalu memanggil `initReservedWords()` dan `initPredefined()`. Sebanyak 32 reserved word dimasukkan ke `tab`, dengan 5 tipe dasar (*integer*, *real*, *boolean*, *char*, dan *string*) diberi `obj = TYPE_DECL` dan `TypeCode` sesuai.

Selain itu, `True` dan `False` dimasukkan sebagai konstanta boolean, sedangkan `Readln` dan `Writeln` dimasukkan sebagai prosedur bawaan. Setelah inisialisasi selesai, `btab[0]` dibuat sebagai scope global, `display[0]` diarahkan ke blok tersebut, dan level diatur menjadi 0.

Method enter() digunakan untuk menambahkan identifier baru ke scope aktif. Entry baru disimpan di `tab` lalu dihubungkan ke linked list identifier pada scope tersebut melalui field `link`. Nilai `last` pada `btab` kemudian diperbarui agar menunjuk ke entry terbaru.

Method lookup() mencari identifier mulai dari scope terdalam hingga scope global. Pencarian dilakukan dengan menelusuri linked list *identifier* pada setiap scope. Jika identifier ditemukan, fungsi mengembalikan indeks entry. Jika tidak, fungsi mengembalikan -1.

Scope baru dibuat menggunakan *pushBlock()*, yang menambahkan blok baru ke `btab`, menaikkan level, dan memperbarui `display`. Sebaliknya, *popBlock()* digunakan untuk keluar dari scope dengan menurunkan level dan menghapus scope aktif dari `display`, tanpa menghapus scope global.

Field *ref* digunakan untuk menghubungkan entry dengan struktur lain, seperti `atab` untuk array atau `btab` untuk procedure dan function. Implementasi juga menyediakan fungsi *dumpTab()*, *dumpBtab()*, *dumpAtab()*, dan *dumpAll()* untuk menampilkan isi tabel simbol.

Saat dekorasi AST, hasil *lookup()* digunakan untuk menambahkan informasi semantik pada node, seperti indeks tabel simbol, tipe data, level, dan blok. Jika *lookup()* mengembalikan -1, compiler dapat melaporkan error *undeclared identifier*, sedangkan pengecekan *redeclaration* dilakukan sebelum *enter()* dipanggil.

2.5 Implementasi Analyzer

- **Entry Point:**

inisiasi `errors`, lalu `visit(root)` untuk traversal.

```
void SemanticAnalyzer::analyze(const NodePtr& root) {
    // entry point semantic analysis
    errors.clear();
    if (!root) {
        reportError("semantic analysis received empty parse
                    tree");
        return;
    }
    visit(root);
}
```

- **Algoritma**

DFS Visitor dengan metode *visit<...>* per non terminal, misal *visitProgram*.

```
SemanticType SemanticAnalyzer::visitProgram(const NodePtr&
node){
    // proses node program utama
    for (const auto& child : node->children){
        visit(child);
    }

    annotate(node, SemanticType{});
    return SemanticType{};
}
```

- **Alur Pemeriksaan**

- **Pendekatan bottom-up per-node**

- setiap *visit** memanggil *visit* pada child dulu, kemudian melakukan pemeriksaan/registrasi/anotasi berdasarkan hasil child

- **Push/pop scope**

- symbols.pushBlock()* / *symbols.popBlock()* saat memasuki/keluar procedure/function/record/compound-statement.

- **Registrasi deklarasi** pada *visitConstDeclaration*, *visitTypeDeclaration*, *visitVarDeclaration*, *visitProcedureDeclaration*, *visitFunctionDeclaration*, dan *visitParameterGroup* panggil *symbols.enter()* untuk menambah tab/btab/atlab.

- **Aturan Semantik yang Diterapkan**

- **Tipe ekspresi**

- **if / while:** kondisi harus boolean (cek dengan *TypeCheck::isBoolean*), jika tidak beri error

- **for:** variabel control harus

- dideklarasikan dan merupakan **VARIABLE**

- bertipe ordinal (*TypeCheck::isOrdinal*)

- ekspresi start/end harus *assignmentCompatible* dengan tipe variabel (cek *TypeCheck::assignmentCompatible*)

- **Assignment:** LHS/RHS dicek dengan

- TypeCheck::assignmentCompatible* (Real $\leftarrow$ Integer, subrange subset, string rules)

- **Pemanggilan prosedur/fungsi:**

- **Lookup symbol;** untuk predefined (detected via *symbols.predefinedCutoff()*)

- **Untuk non-predefined**, ambil parameter formal dari btab (via *entry.ref*) dan cek tiap argumen memakai `TypeCheck::assignmentCompatible`

- **Array/record:**

- **visitArrayType** memanggil `TypeCheck::validArrayIndex` untuk validasi index type (non-real, ordinal/subrange/enumerated).
- **Field access** (.) memeriksa base tipe = record dan lookup field di btab block.

- **Anotasi AST / Parse Tree**

Menyimpan informasi semantik pada node (*tipe, indeks simbol, lexical level, block*) untuk *type checking, error reporting, dan debug*.

```
void SemanticAnalyzer::annotate(const NodePtr& node, const
SemanticType& type, int tabIndex) {
    if (!node) return;
    // simpan hasil analisis ke node parse tree
    node->sem_type = typeName(type);
    node->tab_index = tabIndex;
    node->lev = symbols.currentLevel();
    node->block_index = symbols.currentBlock();
}
```

- **Error handling**

Semua error dikumpulkan ke errors lewat *reportError()*. Analyzer tidak menghentikan traversal pada error pertama, tetapi melaporkan sebanyak mungkin error.

```
void SemanticAnalyzer::reportError(const string& message) {
    errors.push_back("[ERROR] " + message + " !");
}
```

BAB III

PENGUJIAN

1. Test Case 1

Input :

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Output :

```
=== Semantic Analysis ===

--- Decorated AST ---
ProgramNode(name: 'Hello')
+-- Declarations
|   +-- VarDecl('a')  -> type:integer, tab:38, lev:0
|   |   \-- type: 'integer' -> type:integer
|   \-- VarDecl('b')  -> type:integer, tab:39, lev:0
|       \-- type: 'integer' -> type:integer
\-- Block -> type:void, lev:1, blk:1
    +-- Assign(...) -> type:void
    |   +-- target Var('a') -> type:integer, tab:38, lev:0
    |   \-- value Num(5) -> type:integer
    +-- Assign(...) -> type:void
    |   +-- target Var('b') -> type:integer, tab:39, lev:0
    |   \-- value BinOp(op: '+') -> type:integer
    |       +-- left Var('a') -> type:integer, tab:38, lev:0
    |       \-- right Num(10) -> type:integer
    \-- ProcedureCall(name: 'writeln') -> type:void, tab:36, lev:0
        +-- String('Result = ') -> type:string
        \-- Var('b') -> type:integer, tab:39, lev:0

--- Symbol Table (tab) ---
tab:
  idx          id      obj      type  ref  nrm  lev  adr  link
  -----
    0          (empty)  undef      0    0    1    0    0    0
    1            and    undef      0    0    1    0    0    0
    2          array    undef      0    0    1    0    0    1
    3          begin    undef      0    0    1    0    0    2
    4            case    undef      0    0    1    0    0    3
    5          const    undef      0    0    1    0    0    4
    6            div     undef      0    0    1    0    0    5
    7         downto    undef      0    0    1    0    0    6
    8             do     undef      0    0    1    0    0    7
    9           else     undef      0    0    1    0    0    8
```

10	end	undef	0	0	1	0	0	9
11	for	undef	0	0	1	0	0	10
12	function	undef	0	0	1	0	0	11
13	if	undef	0	0	1	0	0	12
14	mod	undef	0	0	1	0	0	13
15	not	undef	0	0	1	0	0	14
16	of	undef	0	0	1	0	0	15
17	or	undef	0	0	1	0	0	16
18	procedure	undef	0	0	1	0	0	17
19	program	undef	0	0	1	0	0	18
20	record	undef	0	0	1	0	0	19
21	repeat	undef	0	0	1	0	0	20
22	integer	type	1	0	1	0	0	21
23	real	type	2	0	1	0	0	22
24	boolean	type	3	0	1	0	0	23
25	char	type	4	0	1	0	0	24
26	string	type	5	0	1	0	0	25
27	then	undef	0	0	1	0	0	26
28	to	undef	0	0	1	0	0	27
29	type	undef	0	0	1	0	0	28
30	until	undef	0	0	1	0	0	29
31	var	undef	0	0	1	0	0	30
32	while	undef	0	0	1	0	0	31
33	true	const	3	0	1	0	1	32
34	false	const	3	0	1	0	0	33
35	readln	proc	0	0	1	0	0	34
36	writeln	proc	0	0	1	0	0	35
37	Hello	program	0	0	1	0	0	36
38	a	var	1	0	1	0	0	37
39	b	var	1	0	1	0	0	38

--- Block Table (btab) ---

```
btab:
  idx   last   lpar   psze   vsze
-----
    0     39     0     0     2
    1      0     0     0     0
```

--- Array Table (atab) ---

```
atab:
(empty)
```

[INFO] Semantic analysis finished successfully.

2. Test Case 2

Input :

```
program AllDecl;

const
  X = 10;
  PI = 3.14;
```

```

type
  T = integer;
  R = record
    a: integer;
    b: real;
  end;
  M = (Jan, Feb, Mar);
  A = array[1..10] of integer;

var
  a: T;
  b: R;
  c: M;
  d: A;

begin
end.

```

Output:

```

=== Semantic Analysis ===

--- Decorated AST ---
ProgramNode(name: 'AllDecl')
+-- Declarations
|   +-- ConstDecl('X')  -> type:integer, tab:38, lev:0
|   |   \-- Num(10)  -> type:integer
|   +-- ConstDecl('PI') -> type:real, tab:39, lev:0
|   |   \-- Num(3.14) -> type:real
|   +-- TypeDecl('T')  -> type:integer, tab:40, lev:0
|   |   \-- type: 'integer' -> type:integer
|   +-- TypeDecl('R')  -> type:record, tab:43, lev:0
|   |   \-- RecordType -> type:record, blk:1
|   |       +-- VarDecl('a')  -> type:integer, tab:41, lev:0
|   |       |   \-- type: 'integer' -> type:integer
|   |       \-- VarDecl('b')  -> type:real, tab:42, lev:0
|   |           \-- type: 'real' -> type:real
|   +-- TypeDecl('M')  -> type:integer, tab:47, lev:0
|   |   \-- EnumType
|   |       +-- Var('Jan')  -> type:integer, tab:44, lev:0
|   |       +-- Var('Feb')  -> type:integer, tab:45, lev:0
|   |       \-- Var('Mar')  -> type:integer, tab:46, lev:0
|   +-- TypeDecl('A')  -> type:array, tab:48, lev:0
|   |   \-- ArrayType
|   |       +-- RangeType
|   |       |   +-- Num(1)  -> type:integer
|   |       |   \-- Num(10) -> type:integer
|   |       \-- type: 'integer' -> type:integer
|   +-- VarDecl('a')  -> type:array, tab:48, lev:0
|   |   \-- type: 'T'  -> type:T
|   +-- VarDecl('b')  -> type:record, tab:49, lev:0
|   |   \-- type: 'R'  -> type:R
|   +-- VarDecl('c')  -> type:integer, tab:50, lev:0
|   |   \-- type: 'M'  -> type:M

```

```
|  \-- VarDecl('d')  -> type:array, tab:51, lev:0
|      \-- type: 'A'  -> type:A
\-- Block  -> type:void, lev:1, blk:1
    \-- Empty  -> type:void
```

--- Symbol Table (tab) ---

tab: idx	id	obj	type	ref	nrm	lev	adr	link
0	(empty)	undef	0	0	1	0	0	0
1	and	undef	0	0	1	0	0	0
2	array	undef	0	0	1	0	0	1
3	begin	undef	0	0	1	0	0	2
4	case	undef	0	0	1	0	0	3
5	const	undef	0	0	1	0	0	4
6	div	undef	0	0	1	0	0	5
7	downto	undef	0	0	1	0	0	6
8	do	undef	0	0	1	0	0	7
9	else	undef	0	0	1	0	0	8
10	end	undef	0	0	1	0	0	9
11	for	undef	0	0	1	0	0	10
12	function	undef	0	0	1	0	0	11
13	if	undef	0	0	1	0	0	12
14	mod	undef	0	0	1	0	0	13
15	not	undef	0	0	1	0	0	14
16	of	undef	0	0	1	0	0	15
17	or	undef	0	0	1	0	0	16
18	procedure	undef	0	0	1	0	0	17
19	program	undef	0	0	1	0	0	18
20	record	undef	0	0	1	0	0	19
21	repeat	undef	0	0	1	0	0	20
22	integer	type	1	0	1	0	0	21
23	real	type	2	0	1	0	0	22
24	boolean	type	3	0	1	0	0	23
25	char	type	4	0	1	0	0	24
26	string	type	5	0	1	0	0	25
27	then	undef	0	0	1	0	0	26
28	to	undef	0	0	1	0	0	27
29	type	undef	0	0	1	0	0	28
30	until	undef	0	0	1	0	0	29
31	var	undef	0	0	1	0	0	30
32	while	undef	0	0	1	0	0	31
33	true	const	3	0	1	0	1	32
34	false	const	3	0	1	0	0	33
35	readln	proc	0	0	1	0	0	34
36	writeln	proc	0	0	1	0	0	35
37	AllDecl	program	0	0	1	0	0	36
38	X	const	1	0	1	0	0	37
39	PI	const	2	0	1	0	0	38
40	T	type	1	0	1	0	0	39
41	a	var	1	0	1	0	0	0
42	b	var	2	0	1	0	0	41
43	R	type	7	1	1	0	0	40
44	Jan	const	1	0	1	0	0	43
45	Feb	const	1	0	1	0	1	44

46		Mar	const	1	0	1	0	2	45
47		M	type	1	0	1	0	0	46
48		A	type	6	1	1	0	0	47
49		b	var	7	1	1	0	0	48
50		c	var	1	0	1	0	0	49
51		d	var	6	1	1	0	0	50

--- Block Table (btab) ---

btab:

idx	last	lpar	psze	vsze

0	51	0	0	3
1	42	0	0	2
2	0	0	0	0

--- Array Table (atab) ---

atab:

idx	xtyp	etyp	eref	low	high	elsz	size

1	1	1	0	1	10	1	10

[INFO] Semantic analysis finished with 1 error(s):

[ERROR] redeclared variable 'a' !

3. Test Case 3

Input:

```

program Control;

var
  a, b, i: integer;

begin
  if a > 0 then
    b := 1
  else
    b := 0;

  case a of
    1: b := 1;
    2, 3: b := 2;
  end;

  while a < 10 do
    begin
      a := a + 1;
    end;

  repeat
    a := a - 1;
  until a = 0;

```

```

for i := 1 to 10 do
begin
  a := i;
end;
end.

```

Output:

```

=== Semantic Analysis ===

--- Decorated AST ---
ProgramNode(name: 'Control')
+-- Declarations
|   +-- VarDecl('a') -> type:integer, tab:38, lev:0
|   |   \-- type: 'integer' -> type:integer
|   +-- VarDecl('b') -> type:integer, tab:39, lev:0
|   |   \-- type: 'integer' -> type:integer
|   \-- VarDecl('i') -> type:integer, tab:40, lev:0
|       \-- type: 'integer' -> type:integer
\-- Block -> type:void, lev:1, blk:1
    +-- If -> type:void
    |   +-- BinOp(op: '>') -> type:boolean
    |   |   +-- left Var('a') -> type:integer, tab:38, lev:0
    |   |   \-- right Num(0) -> type:integer
    |   +-- Assign(...) -> type:void
    |   |   +-- target Var('b') -> type:integer, tab:39, lev:0
    |   |   \-- value Num(1) -> type:integer
    |   \-- Assign(...) -> type:void
    |       +-- target Var('b') -> type:integer, tab:39, lev:0
    |       \-- value Num(0) -> type:integer
    +-- Case -> type:void
    |   +-- Var('a') -> type:integer, tab:38, lev:0
    |   \-- Block
    |       +-- Num(1) -> type:integer
    |       \-- Assign(...) -> type:void
    |           +-- target Var('b') -> type:integer, tab:39, lev:0
    |           \-- value Num(1) -> type:integer
    +-- While -> type:void
    |   +-- BinOp(op: '<') -> type:boolean
    |   |   +-- left Var('a') -> type:integer, tab:38, lev:0
    |   |   \-- right Num(10) -> type:integer
    |   \-- Block -> type:void, lev:2, blk:2
    |       \-- Assign(...) -> type:void
    |           +-- target Var('a') -> type:integer, tab:38, lev:0
    |           \-- value BinOp(op: '+') -> type:integer
    |               +-- left Var('a') -> type:integer, tab:38, lev:0
    |               \-- right Num(1) -> type:integer
    +-- Repeat -> type:void
    |   +-- Block -> type:void, lev:2, blk:2
    |   |   \-- Assign(...) -> type:void
    |   |       +-- target Var('a') -> type:integer, tab:38, lev:0
    |   |       \-- value BinOp(op: '-') -> type:integer
    |   |           +-- left Var('a') -> type:integer, tab:38, lev:0
    |   |           \-- right Num(1) -> type:integer
    |   \-- BinOp(op: '=') -> type:boolean

```

```

|      +-- left Var('a')  -> type:integer, tab:38, lev:0
|      \-- right Num(0)   -> type:integer
\-- For(var: 'i', dir: 'to') -> type:void
    +-- Num(1)  -> type:integer
    +-- Num(10) -> type:integer
    \-- Block  -> type:void, lev:2, blk:2
        \-- Assign(...) -> type:void
            +-- target Var('a') -> type:integer, tab:38, lev:0
            \-- value Var('i')  -> type:integer, tab:40, lev:0

```

--- Symbol Table (tab) ---

tab: idx	id	obj	type	ref	nrm	lev	adr	link
0	(empty)	undef	0	0	1	0	0	0
1	and	undef	0	0	1	0	0	0
2	array	undef	0	0	1	0	0	1
3	begin	undef	0	0	1	0	0	2
4	case	undef	0	0	1	0	0	3
5	const	undef	0	0	1	0	0	4
6	div	undef	0	0	1	0	0	5
7	downto	undef	0	0	1	0	0	6
8	do	undef	0	0	1	0	0	7
9	else	undef	0	0	1	0	0	8
10	end	undef	0	0	1	0	0	9
11	for	undef	0	0	1	0	0	10
12	function	undef	0	0	1	0	0	11
13	if	undef	0	0	1	0	0	12
14	mod	undef	0	0	1	0	0	13
15	not	undef	0	0	1	0	0	14
16	of	undef	0	0	1	0	0	15
17	or	undef	0	0	1	0	0	16
18	procedure	undef	0	0	1	0	0	17
19	program	undef	0	0	1	0	0	18
20	record	undef	0	0	1	0	0	19
21	repeat	undef	0	0	1	0	0	20
22	integer	type	1	0	1	0	0	21
23	real	type	2	0	1	0	0	22
24	boolean	type	3	0	1	0	0	23
25	char	type	4	0	1	0	0	24
26	string	type	5	0	1	0	0	25
27	then	undef	0	0	1	0	0	26
28	to	undef	0	0	1	0	0	27
29	type	undef	0	0	1	0	0	28
30	until	undef	0	0	1	0	0	29
31	var	undef	0	0	1	0	0	30
32	while	undef	0	0	1	0	0	31
33	true	const	3	0	1	0	1	32
34	false	const	3	0	1	0	0	33
35	readln	proc	0	0	1	0	0	34
36	writeln	proc	0	0	1	0	0	35
37	Control	program	0	0	1	0	0	36
38	a	var	1	0	1	0	0	37
39	b	var	1	0	1	0	0	38
40	i	var	1	0	1	0	0	39

```

--- Block Table (btab) ---

btab:
  idx   last   lpar   psze   vsze
-----
    0     40     0     0     3
    1      0     0     0     0
    2      0     0     0     0
    3      0     0     0     0

--- Array Table (atab) ---

atab:
(empty)

[INFO] Semantic analysis finished successfully.

```

4. Test Case 4

Input:

```

program Subprograms;

procedure Swap(a, b: integer);
var
  temp: integer;
begin
  temp := a;
  a := b;
  b := temp;
end;

function Add(a, b: integer): integer;
begin
  Add := a + b;
end;

begin
  Swap(a, b);
  c := Add(1, 2);
end.

```

Output:

```

=== Semantic Analysis ===

--- Decorated AST ---
ProgramNode(name: 'Subprograms')
+-- Declarations
|   +-- SubprogramDecl('Swap')  -> type:unknown, tab:38, lev:0
|   |   |-- Block -> type:void, lev:1, blk:1
|   |       +-- Assign(...) -> type:void
|   |           |   +-- target Var('temp')

```

```

|      |      |      \-- value Var('a')
|      |      +-- Assign(...) -> type:void
|      |      |      +-- target Var('a')
|      |      |      \-- value Var('b')
|      |      \-- Assign(...) -> type:void
|      |      +-- target Var('b')
|      |      \-- value Var('temp')
|      \-- SubprogramDecl('Add') -> type:integer, tab:42, lev:0
|      \-- Block -> type:void, lev:1, blk:1
|      \-- Assign(...) -> type:void
|      +-- target Var('Add') -> type:integer, tab:42, lev:0
|      \-- value BinOp(op: '+') -> type:integer
|      +-- left Var('a')
|      \-- right Var('b')
\-- Block -> type:void, lev:1, blk:1
+-- ProcedureCall(name: 'Swap') -> type:void, tab:38, lev:0
| +-- Var('a')
| \-- Var('b')
\-- Assign(...) -> type:void
+-- target Var('c')
\-- value FunctionCall(name: 'Add') -> type:integer, tab:42,
lev:0
+-- Num(1) -> type:integer
\-- Num(2) -> type:integer

```

--- Symbol Table (tab) ---

tab: idx	id	obj	type	ref	nrm	lev	adr	link
0	(empty)	undef	0	0	1	0	0	0
1	and	undef	0	0	1	0	0	0
2	array	undef	0	0	1	0	0	1
3	begin	undef	0	0	1	0	0	2
4	case	undef	0	0	1	0	0	3
5	const	undef	0	0	1	0	0	4
6	div	undef	0	0	1	0	0	5
7	downto	undef	0	0	1	0	0	6
8	do	undef	0	0	1	0	0	7
9	else	undef	0	0	1	0	0	8
10	end	undef	0	0	1	0	0	9
11	for	undef	0	0	1	0	0	10
12	function	undef	0	0	1	0	0	11
13	if	undef	0	0	1	0	0	12
14	mod	undef	0	0	1	0	0	13
15	not	undef	0	0	1	0	0	14
16	of	undef	0	0	1	0	0	15
17	or	undef	0	0	1	0	0	16
18	procedure	undef	0	0	1	0	0	17
19	program	undef	0	0	1	0	0	18
20	record	undef	0	0	1	0	0	19
21	repeat	undef	0	0	1	0	0	20
22	integer	type	1	0	1	0	0	21
23	real	type	2	0	1	0	0	22
24	boolean	type	3	0	1	0	0	23
25	char	type	4	0	1	0	0	24

26	string	type	5	0	1	0	0	25
27	then	undef	0	0	1	0	0	26
28	to	undef	0	0	1	0	0	27
29	type	undef	0	0	1	0	0	28
30	until	undef	0	0	1	0	0	29
31	var	undef	0	0	1	0	0	30
32	while	undef	0	0	1	0	0	31
33	true	const	3	0	1	0	1	32
34	false	const	3	0	1	0	0	33
35	readln	proc	0	0	1	0	0	34
36	writeln	proc	0	0	1	0	0	35
37	Subprograms	program	0	0	1	0	0	36
38	Swap	proc	0	1	1	0	0	37
39	a	var	1	0	1	1	0	0
40	b	var	1	0	1	1	0	39
41	temp	var	1	0	1	1	0	40
42	Add	func	1	3	1	0	0	38
43	a	var	1	0	1	1	0	0
44	b	var	1	0	1	1	0	43

--- Block Table (btab) ---

btab:

idx	last	lpar	psze	vsze

0	42	0	0	0
1	41	40	2	3
2	0	0	0	0
3	44	44	2	2
4	0	0	0	0
5	0	0	0	0

--- Array Table (atab) ---

atab:

(empty)

[INFO] Semantic analysis finished with 3 error(s):

[ERROR] undeclared identifier 'a' !

[ERROR] undeclared identifier 'b' !

[ERROR] undeclared identifier 'c' !

5. Test Case 5

Input:

```

program Expr;

var
  a, b, c, d: integer;

begin
  a := 1 + 2 * 3 - 4 div 5 + 6 mod 7;
  b := a * (b + c);
  c := not a and b or c;

```

```
d := (a = b) and (c <> d);
end.
```

Output:

```
=== Semantic Analysis ===

--- Decorated AST ---
ProgramNode(name: 'Expr')
+-- Declarations
|   +-- VarDecl('a') -> type:integer, tab:38, lev:0
|   |   \-- type: 'integer' -> type:integer
|   +-- VarDecl('b') -> type:integer, tab:39, lev:0
|   |   \-- type: 'integer' -> type:integer
|   +-- VarDecl('c') -> type:integer, tab:40, lev:0
|   |   \-- type: 'integer' -> type:integer
|   \-- VarDecl('d') -> type:integer, tab:41, lev:0
|       \-- type: 'integer' -> type:integer
\-- Block -> type:void, lev:1, blk:1
    +-- Assign(...) -> type:void
    |   +-- target Var('a') -> type:integer, tab:38, lev:0
    |   \-- value BinOp(op: '+') -> type:integer
    |       +-- left BinOp(op: '-') -> type:integer
    |       |   +-- left BinOp(op: '+') -> type:integer
    |       |   |   +-- left Num(1) -> type:integer
    |       |   |   \-- right BinOp(op: '*') -> type:integer
    |       |   |       +-- left Num(2) -> type:integer
    |       |   |       \-- right Num(3) -> type:integer
    |       |   \-- right BinOp(op: 'div') -> type:integer
    |       |       +-- left Num(4) -> type:integer
    |       |       \-- right Num(5) -> type:integer
    |       \-- right BinOp(op: 'mod') -> type:integer
    |           +-- left Num(6) -> type:integer
    |           \-- right Num(7) -> type:integer
    +-- Assign(...) -> type:void
    |   +-- target Var('b') -> type:integer, tab:39, lev:0
    |   \-- value BinOp(op: '*') -> type:integer
    |       +-- left Var('a') -> type:integer, tab:38, lev:0
    |       \-- right BinOp(op: '+') -> type:integer
    |           +-- left Var('b') -> type:integer, tab:39, lev:0
    |           \-- right Var('c') -> type:integer, tab:40, lev:0
    +-- Assign(...) -> type:void
    |   +-- target Var('c') -> type:integer, tab:40, lev:0
    |   \-- value BinOp(op: 'or') -> type:boolean
    |       +-- left BinOp(op: 'and') -> type:boolean
    |       |   +-- left UnaryOp(op: 'not') -> type:boolean
    |       |   |   \-- operand Var('a') -> type:integer, tab:38, lev:0
    |       |   \-- right Var('b') -> type:integer, tab:39, lev:0
    |       \-- right Var('c') -> type:integer, tab:40, lev:0
    \-- Assign(...) -> type:void
    |   +-- target Var('d') -> type:integer, tab:41, lev:0
    |   \-- value BinOp(op: 'and') -> type:boolean
    |       +-- left BinOp(op: '=') -> type:boolean
    |       |   +-- left Var('a') -> type:integer, tab:38, lev:0
    |       |   \-- right Var('b') -> type:integer, tab:39, lev:0
```

```

\-- right BinOp(op: '<>') -> type:boolean
+-- left Var('c') -> type:integer, tab:40, lev:0
\-- right Var('d') -> type:integer, tab:41, lev:0

```

--- Symbol Table (tab) ---

tab:

idx	id	obj	type	ref	nrm	lev	adr	link
0	(empty)	undef	0	0	1	0	0	0
1	and	undef	0	0	1	0	0	0
2	array	undef	0	0	1	0	0	1
3	begin	undef	0	0	1	0	0	2
4	case	undef	0	0	1	0	0	3
5	const	undef	0	0	1	0	0	4
6	div	undef	0	0	1	0	0	5
7	downto	undef	0	0	1	0	0	6
8	do	undef	0	0	1	0	0	7
9	else	undef	0	0	1	0	0	8
10	end	undef	0	0	1	0	0	9
11	for	undef	0	0	1	0	0	10
12	function	undef	0	0	1	0	0	11
13	if	undef	0	0	1	0	0	12
14	mod	undef	0	0	1	0	0	13
15	not	undef	0	0	1	0	0	14
16	of	undef	0	0	1	0	0	15
17	or	undef	0	0	1	0	0	16
18	procedure	undef	0	0	1	0	0	17
19	program	undef	0	0	1	0	0	18
20	record	undef	0	0	1	0	0	19
21	repeat	undef	0	0	1	0	0	20
22	integer	type	1	0	1	0	0	21
23	real	type	2	0	1	0	0	22
24	boolean	type	3	0	1	0	0	23
25	char	type	4	0	1	0	0	24
26	string	type	5	0	1	0	0	25
27	then	undef	0	0	1	0	0	26
28	to	undef	0	0	1	0	0	27
29	type	undef	0	0	1	0	0	28
30	until	undef	0	0	1	0	0	29
31	var	undef	0	0	1	0	0	30
32	while	undef	0	0	1	0	0	31
33	true	const	3	0	1	0	1	32
34	false	const	3	0	1	0	0	33
35	readln	proc	0	0	1	0	0	34
36	writeln	proc	0	0	1	0	0	35
37	Expr	program	0	0	1	0	0	36
38	a	var	1	0	1	0	0	37
39	b	var	1	0	1	0	0	38
40	c	var	1	0	1	0	0	39
41	d	var	1	0	1	0	0	40

--- Block Table (btab) ---

btab:

idx	last	lpar	psze	vsze
-----	------	------	------	------

```

-----
  0      41      0      0      4
  1       0      0      0      0

--- Array Table (atab) ---

atab:
(empty)

[INFO] Semantic analysis finished with 5 error(s):
  [ERROR] not operand must be boolean !
  [ERROR] operator 'and' requires boolean operands, got 'boolean' and
'integer' !
  [ERROR] operator 'or' requires boolean operands, got 'boolean' and
'integer' !
  [ERROR] assignment incompatible: cannot assign 'boolean' to 'integer'
!
  [ERROR] assignment incompatible: cannot assign 'boolean' to 'integer'
!

```

6. Test Case 6

Input:

```

program ArrayDemo;

var
  arr: array [1..5] of integer;
  i, sum: integer;

begin
  i := 1;
  sum := 0;
  arr[1] := 10;
  arr[2] := arr[1] + 20;
  sum := arr[1] + arr[2];
end.

```

Output:

```

=== Abstract Syntax Tree ===
ProgramNode(name: 'ArrayDemo')
+-- Declarations
|   +-- VarDecl('arr')
|   |   \-- ArrayType
|   |       +-- RangeType
|   |           |   +-- Num(1)
|   |           |   \-- Num(5)
|   |           \-- type: 'integer'
|   +-- VarDecl('i')
|   |   \-- type: 'integer'
|   \-- VarDecl('sum')
|       \-- type: 'integer'
\-- Block

```

```

+-- Assign(...)
|   +-- target Var('i')
|   \-- value Num(1)
+-- Assign(...)
|   +-- target Var('sum')
|   \-- value Num(0)
+-- Assign(...)
|   +-- target IndexAccess(base: Var('arr'), index: null)
|   |   \-- Var('arr')
|   \-- value Num(10)
+-- Assign(...)
|   +-- target IndexAccess(base: Var('arr'), index: null)
|   |   \-- Var('arr')
|   \-- value BinOp(op: '+')
|       +-- left IndexAccess(base: Var('arr'), index: null)
|       |   \-- Var('arr')
|       \-- right Num(20)
\-- Assign(...)
    +-- target Var('sum')
    \-- value BinOp(op: '+')
        +-- left IndexAccess(base: Var('arr'), index: null)
        |   \-- Var('arr')
        \-- right IndexAccess(base: Var('arr'), index: null)
            \-- Var('arr')

```

[INFO] AST build finished successfully.

Export AST to file ? [N/y]

N

=== Semantic Analysis ===

--- Decorated AST ---

ProgramNode(name: 'ArrayDemo')

+-- Declarations

```

|   +-- VarDecl('arr') -> type:array, tab:38, lev:0
|   |   \-- ArrayType
|   |       +-- RangeType
|   |       |   +-- Num(1) -> type:integer
|   |       |   \-- Num(5) -> type:integer
|   |       \-- type: 'integer' -> type:integer
|   +-- VarDecl('i') -> type:integer, tab:39, lev:0
|   |   \-- type: 'integer' -> type:integer
|   \-- VarDecl('sum') -> type:integer, tab:40, lev:0
|       \-- type: 'integer' -> type:integer
\-- Block -> type:void, lev:1, blk:1
    +-- Assign(...) -> type:void
    |   +-- target Var('i') -> type:integer, tab:39, lev:0
    |   \-- value Num(1) -> type:integer
    +-- Assign(...) -> type:void
    |   +-- target Var('sum') -> type:integer, tab:40, lev:0
    |   \-- value Num(0) -> type:integer
    +-- Assign(...) -> type:void
    |   +-- target IndexAccess(base: Var('arr'), index: null)
    |   |   \-- Var('arr') -> type:array, tab:38, lev:0
    |   \-- value Num(10) -> type:integer
    +-- Assign(...) -> type:void

```

```

|   +-- target IndexAccess(base: Var('arr'), index: null)
|   |   \-- Var('arr')  -> type:array, tab:38, lev:0
|   \-- value BinOp(op: '+')  -> type:integer
|       +-- left IndexAccess(base: Var('arr'), index: null)
|       |   \-- Var('arr')  -> type:array, tab:38, lev:0
|       \-- right Num(20)  -> type:integer
\-- Assign(...)  -> type:void
    +-- target Var('sum')  -> type:integer, tab:40, lev:0
    \-- value BinOp(op: '+')  -> type:integer
        +-- left IndexAccess(base: Var('arr'), index: null)
        |   \-- Var('arr')  -> type:array, tab:38, lev:0
        \-- right IndexAccess(base: Var('arr'), index: null)
            \-- Var('arr')  -> type:array, tab:38, lev:0

```

--- Symbol Table (tab) ---

tab:								
idx	id	obj	type	ref	nrm	lev	adr	link
0	(empty)	undef	0	0	1	0	0	0
1	and	undef	0	0	1	0	0	0
2	array	undef	0	0	1	0	0	1
3	begin	undef	0	0	1	0	0	2
4	case	undef	0	0	1	0	0	3
5	const	undef	0	0	1	0	0	4
6	div	undef	0	0	1	0	0	5
7	downto	undef	0	0	1	0	0	6
8	do	undef	0	0	1	0	0	7
9	else	undef	0	0	1	0	0	8
10	end	undef	0	0	1	0	0	9
11	for	undef	0	0	1	0	0	10
12	function	undef	0	0	1	0	0	11
13	if	undef	0	0	1	0	0	12
14	mod	undef	0	0	1	0	0	13
15	not	undef	0	0	1	0	0	14
16	of	undef	0	0	1	0	0	15
17	or	undef	0	0	1	0	0	16
18	procedure	undef	0	0	1	0	0	17
19	program	undef	0	0	1	0	0	18
20	record	undef	0	0	1	0	0	19
21	repeat	undef	0	0	1	0	0	20
22	integer	type	1	0	1	0	0	21
23	real	type	2	0	1	0	0	22
24	boolean	type	3	0	1	0	0	23
25	char	type	4	0	1	0	0	24
26	string	type	5	0	1	0	0	25
27	then	undef	0	0	1	0	0	26
28	to	undef	0	0	1	0	0	27
29	type	undef	0	0	1	0	0	28
30	until	undef	0	0	1	0	0	29
31	var	undef	0	0	1	0	0	30
32	while	undef	0	0	1	0	0	31
33	true	const	3	0	1	0	1	32
34	false	const	3	0	1	0	0	33
35	readln	proc	0	0	1	0	0	34
36	writeln	proc	0	0	1	0	0	35

37	ArrayDemo	program	0	0	1	0	0	36
38	arr	var	6	1	1	0	0	37
39	i	var	1	0	1	0	0	38
40	sum	var	1	0	1	0	0	39

--- Block Table (btab) ---

btab:

idx	last	lpar	psze	vsze

0	40	0	0	3
1	0	0	0	0

--- Array Table (atab) ---

atab:

idx	xtyp	etyp	eref	low	high	elsz	size

1	1	1	0	1	5	1	5

BAB IV

KESIMPULAN DAN SARAN

1. Kesimpulan

Pada milestone ketiga dari tugas besar ini telah berhasil dirancang dan diimplementasikan bagian Semantic Analysis untuk compiler-interpreter bahasa Arion. Tahap ini melanjutkan hasil dari milestone sebelumnya dengan mengubah *Parse Tree* menjadi *Abstract Syntax Tree* (AST), kemudian melakukan pengubahan AST menjadi *decorated* AST untuk menambahkan informasi semantik.

Implementasi semantic analysis mencakup pembentukan AST, pengelolaan Symbol Table, pendefinisian predefined identifier, serta perancangan aturan kompatibilitas tipe. Symbol Table digunakan untuk menyimpan informasi identifier, tipe data, scope, dan kelas objek, sehingga compiler dapat melakukan pencarian identifier dan mendukung deteksi kesalahan seperti identifier yang belum dideklarasikan atau deklarasi ulang.

Selain itu, aturan type compatibility telah dirancang untuk memeriksa kesesuaian tipe pada assignment, ekspresi, operasi aritmatika, operasi logika, serta struktur kontrol program. Dengan demikian, milestone ini telah berhasil membangun dasar semantic analysis yang memastikan program tidak hanya benar secara sintaksis, tetapi juga konsisten secara semantik.

2. Saran

Beberapa saran yang bisa diberikan selama pengerjaan ini adalah:

Kepada Kelompok:

- Melakukan perencanaan matang yang lebih runut sehingga mudah dalam melakukan pengerjaan
- Mengerjakan finalisasi program jauh-jauh hari sehingga menghindari terlewatnya kasus test case dari sebuah masalah

Kepada Asisten

- Memastikan spesifikasi lebih jelas sehingga mengurangi jumlah revisi yang harus diterapkan dari QnA
- Memberikan contoh format error handling yang diharapkan sehingga implementasi recovery mode dapat lebih konsisten juga terarah.
- Menyediakan lebih banyak testcase referensi terutama untuk kasus ambigu dan kompleks sehingga proses pengujian lebih mendalam.
- Memberikan arahan yang lebih jelas agar memudahkan implementasi program

LAMPIRAN

Link Github

<https://github.com/Nusss0/CNA-Tubes-IF2224-2026>

Pembagian Tugas

13524020	25%
13524080	25%
13524104	25%
13524106	25%

REFERENSI

- C, B. P. (2021, December 6). *Abstract Syntax Tree (AST) - Explained in Plain English*. DEV Community. Accessed May 21, 2026, from <https://dev.to/balapriya/abstract-syntax-tree-ast-explained-in-plain-english-1h38>
- Symbol Table in Compiler*. (2026, April 21). GeeksforGeeks. Accessed May 21, 2026, from <https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>
- Understanding Semantic Analysis - NLP*. (2021, November 28). GeeksforGeeks. Accessed May 21, 2026, from <https://www.geeksforgeeks.org/nlp/understanding-semantic-analysis-nlp/>
- Worcester Polytechnic Institute - Computer Science. (n.d.). *6.5.2 L-Attributed Attribute Grammars*. Programming Language Translations. Accessed May 21, 2026, from <https://web.cs.wpi.edu/~cs544/PLT6.5.2.html>